

Week 5 Gate 5 Demo Report

Fixtures:

A fixtures folder is created to store a TypeScript file called evidence.ts. This file is used to attach evidence artifacts to the playwright reports.

Playwright.config.ts:

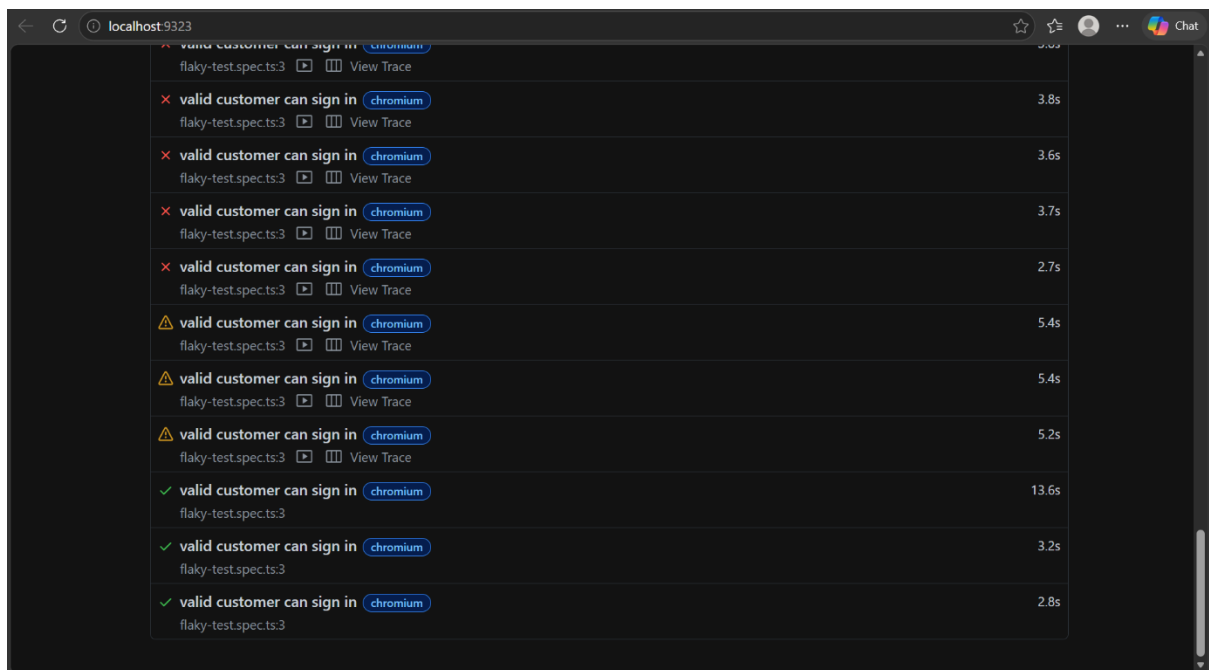
In playwright.config.ts, retries is set to 1, trace is set to be generated everytime, screenshots only on failure, and videos on first retries.

1) Flaky Test

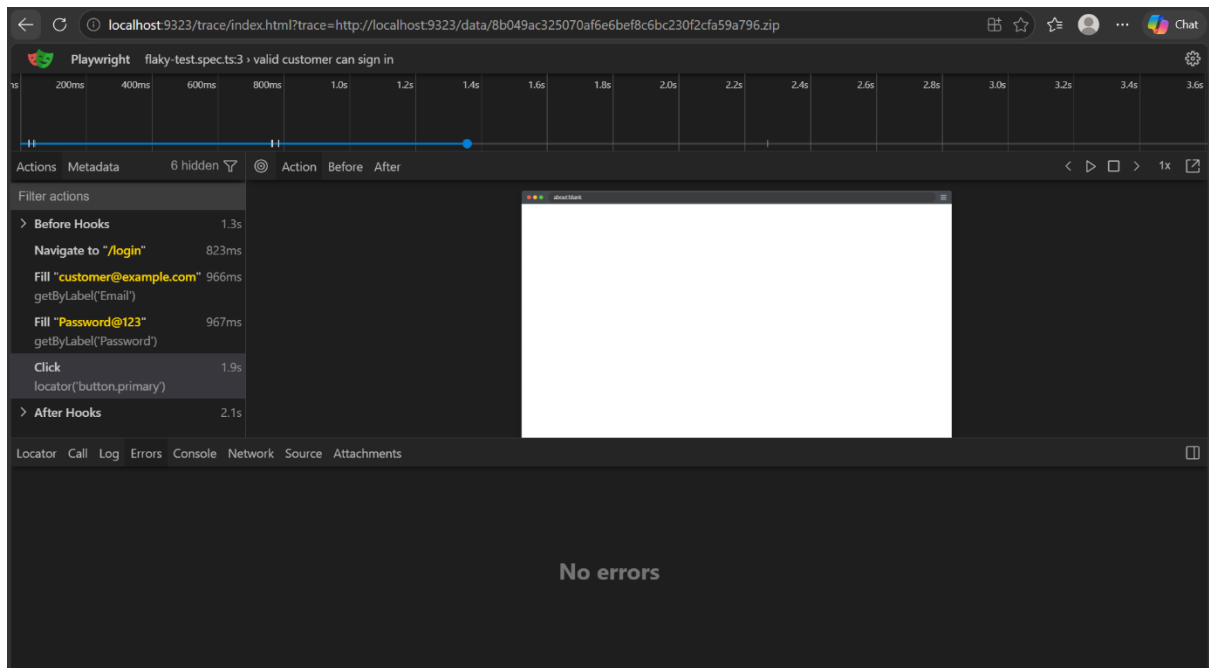
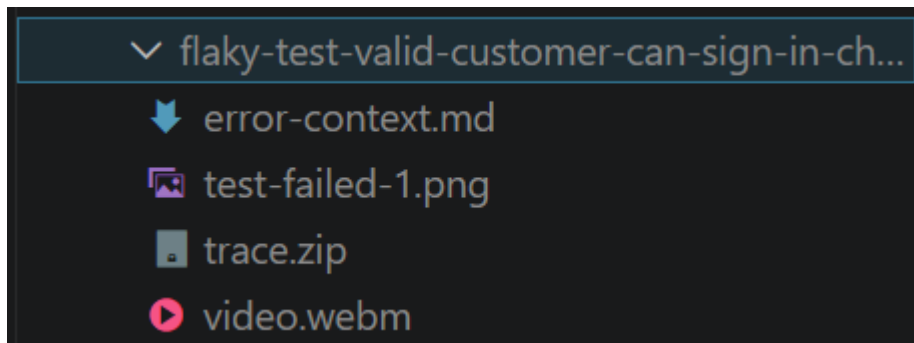
Created a test called flaky-test.spec.ts in which we attempt to click 'Sign-in' button on login page without waiting for it to be enabled, resulting in flaky results from the test.

The test is repeated 50 times with retries set to 1 in case of failure.

Out of 50, only 3 passed, 3 were labelled as flaky tests and the rest 44 failed.



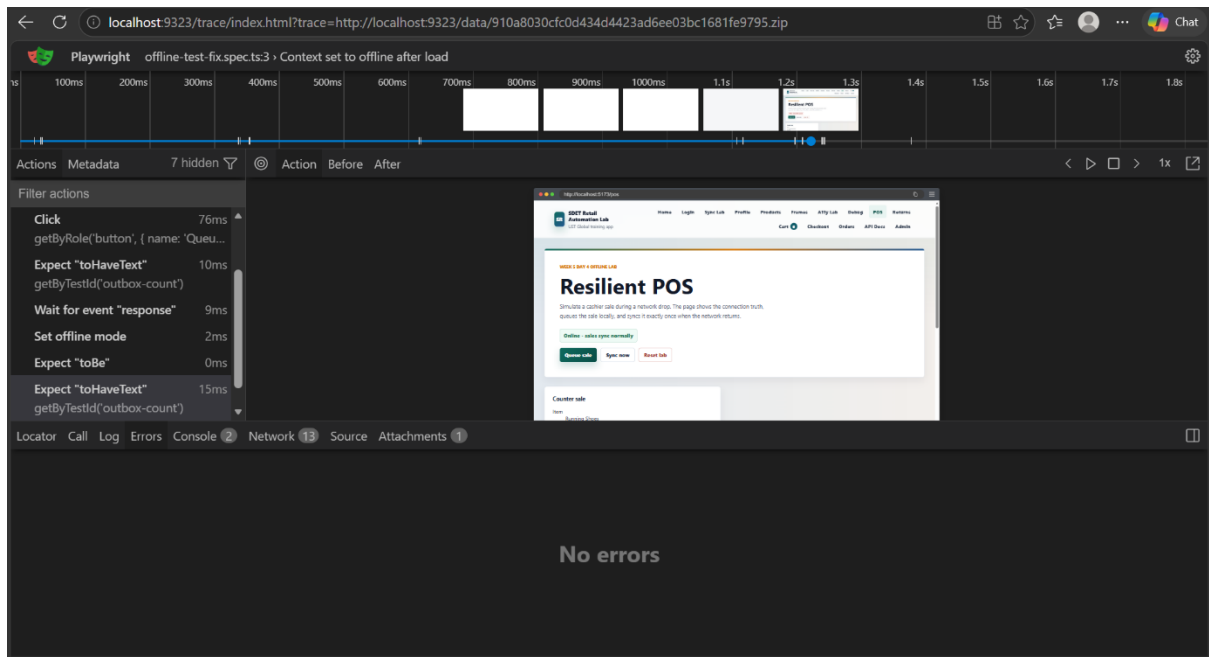
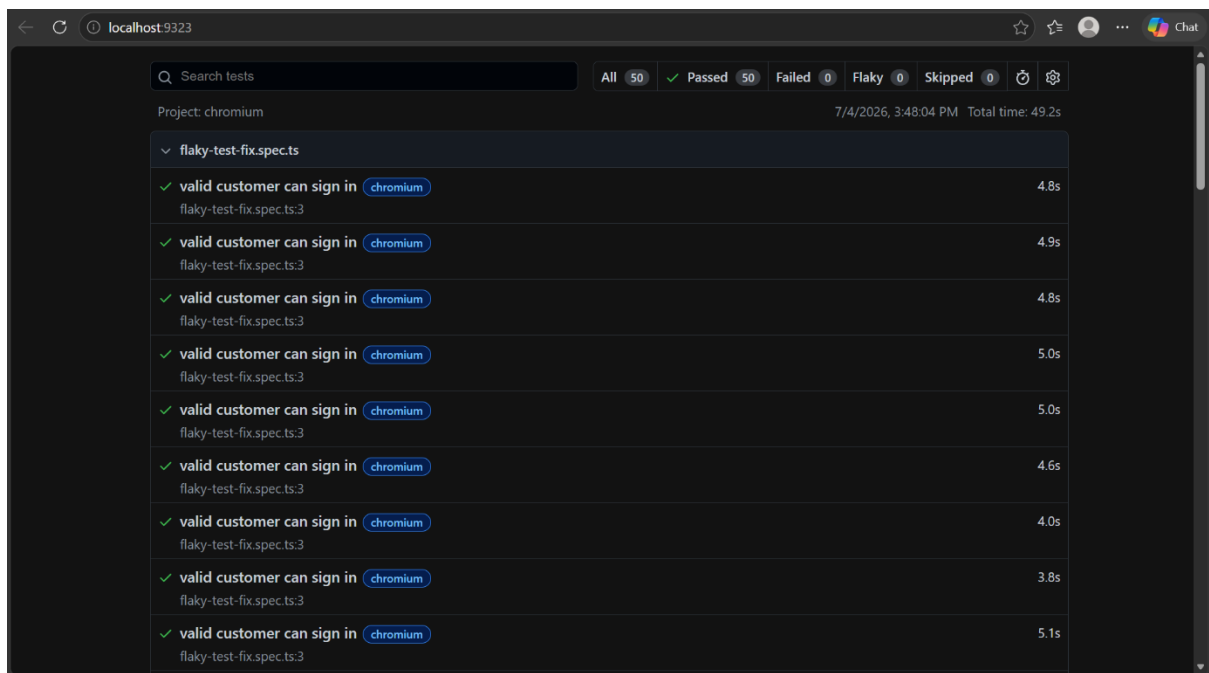
Failed cases generate a screenshot, video and trace file for debugging as shown.



The test was fixed by using web first awaits to wait for the button to be enabled automatically before clicking.

The fixed test case was created in a file called flaky-test-fix.spec.ts and was repeated 50 times as well.

After inserting await before each step, the test case passed 50 times and traces were generated for each.



Commands:

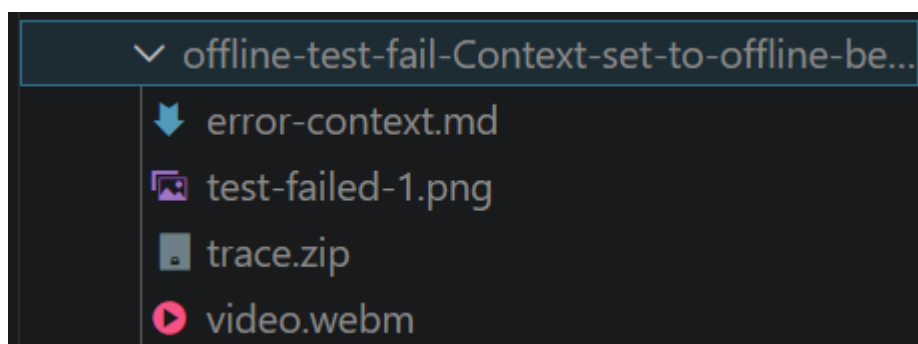
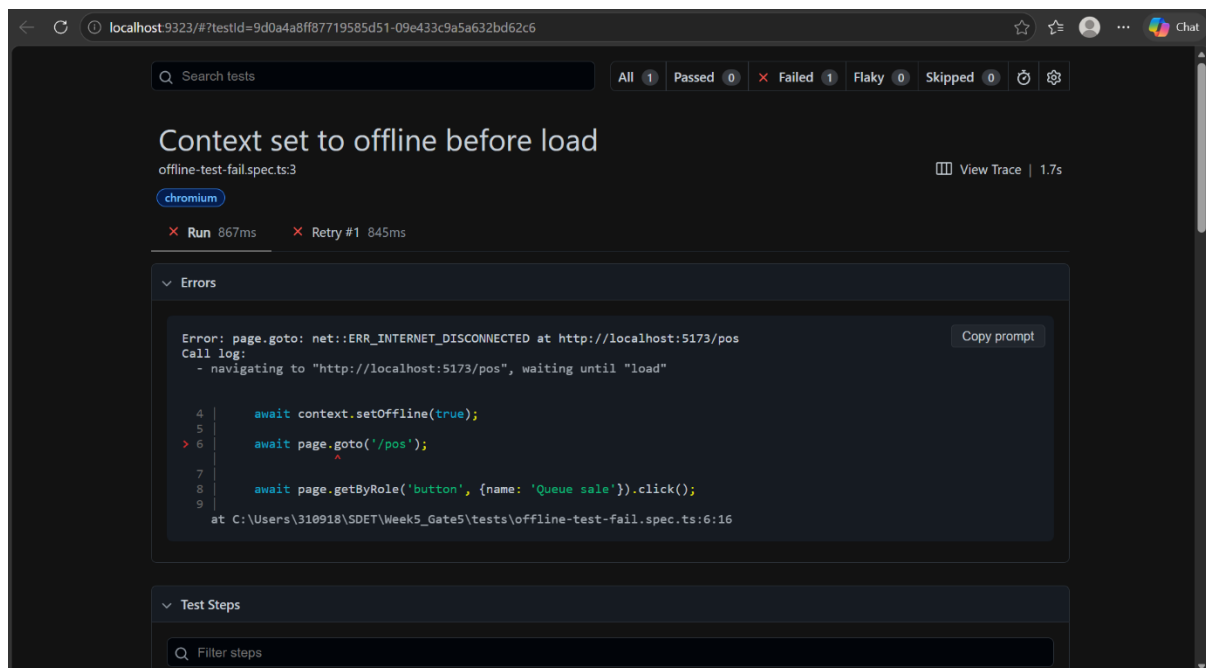
`npx playwright test tests/flaky-test.spec.ts --repeat-each=50`

`npx playwright test tests/flaky-test-fix.spec.ts --repeat-each=50`

`npx playwright show-report`

2) Offline-Sync test

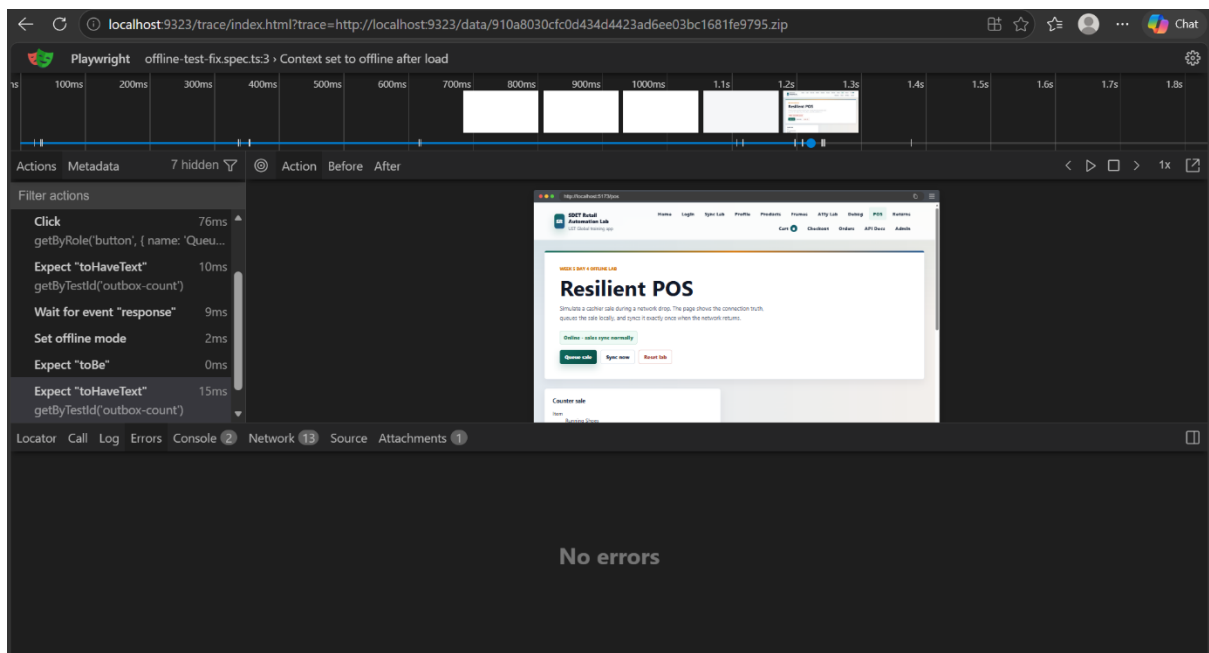
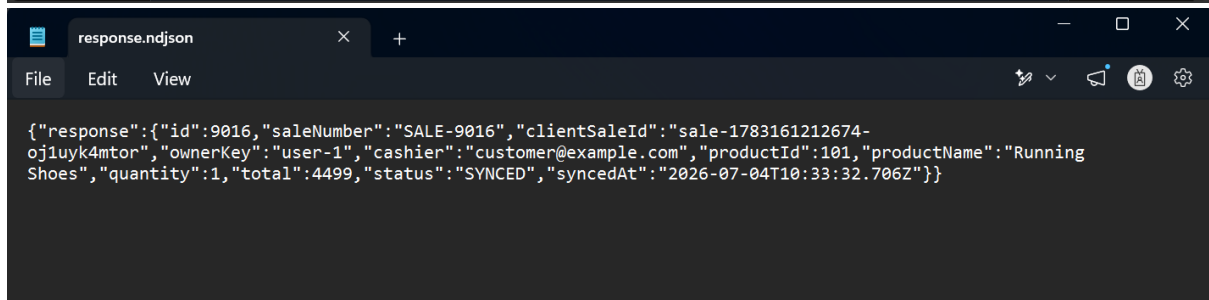
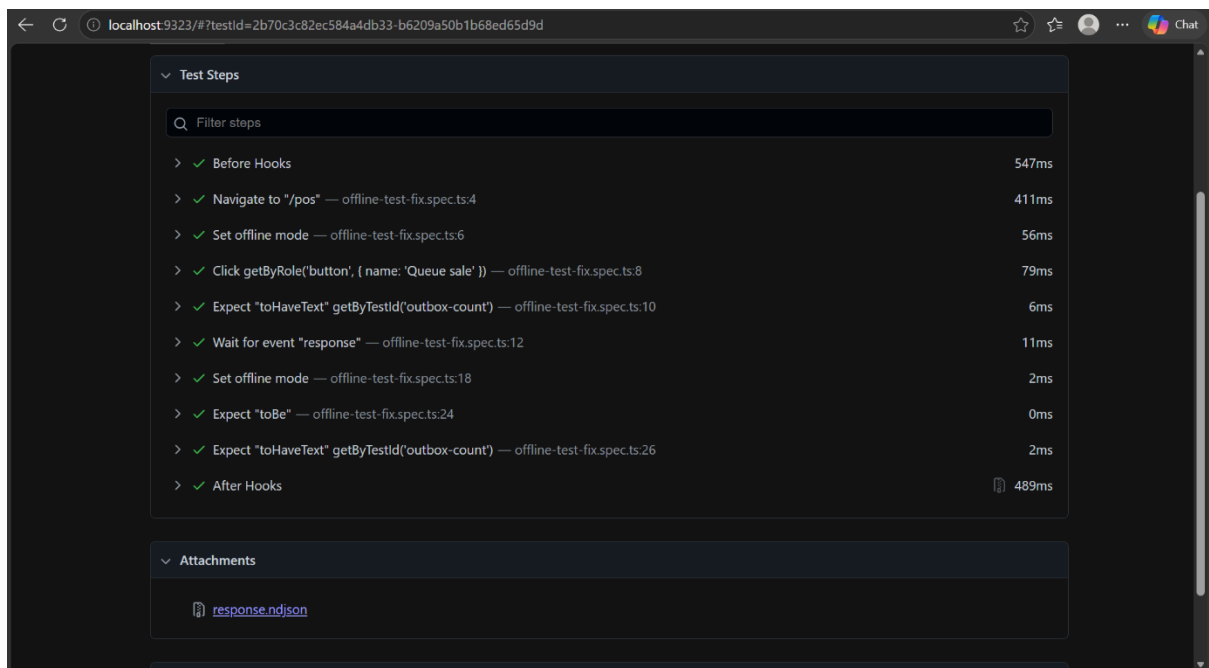
A test file called `offline-test-fail.spec.ts` was created to demonstrate syncing requests offline. In this test, the test fails due to the context being set to offline before the page loads. Screenshot, video and trace are generated upon failure.



A second test file called `offline-test-fix.spec.ts` is created to fix this error by setting the context to offline after the page loads. After the context is set to offline, the 'queue sale' button is clicked once and outbox count is asserted to be 1. We await the request's response as a promise.

Once the browser context is set back to online, the response is received and is asserted to have the status=SYNCED and the outbox count is asserted to be 0.

Upon success, the response body json is attached as evidence in the report and can be downloaded. A trace is also created for the fixed test.



Commands:

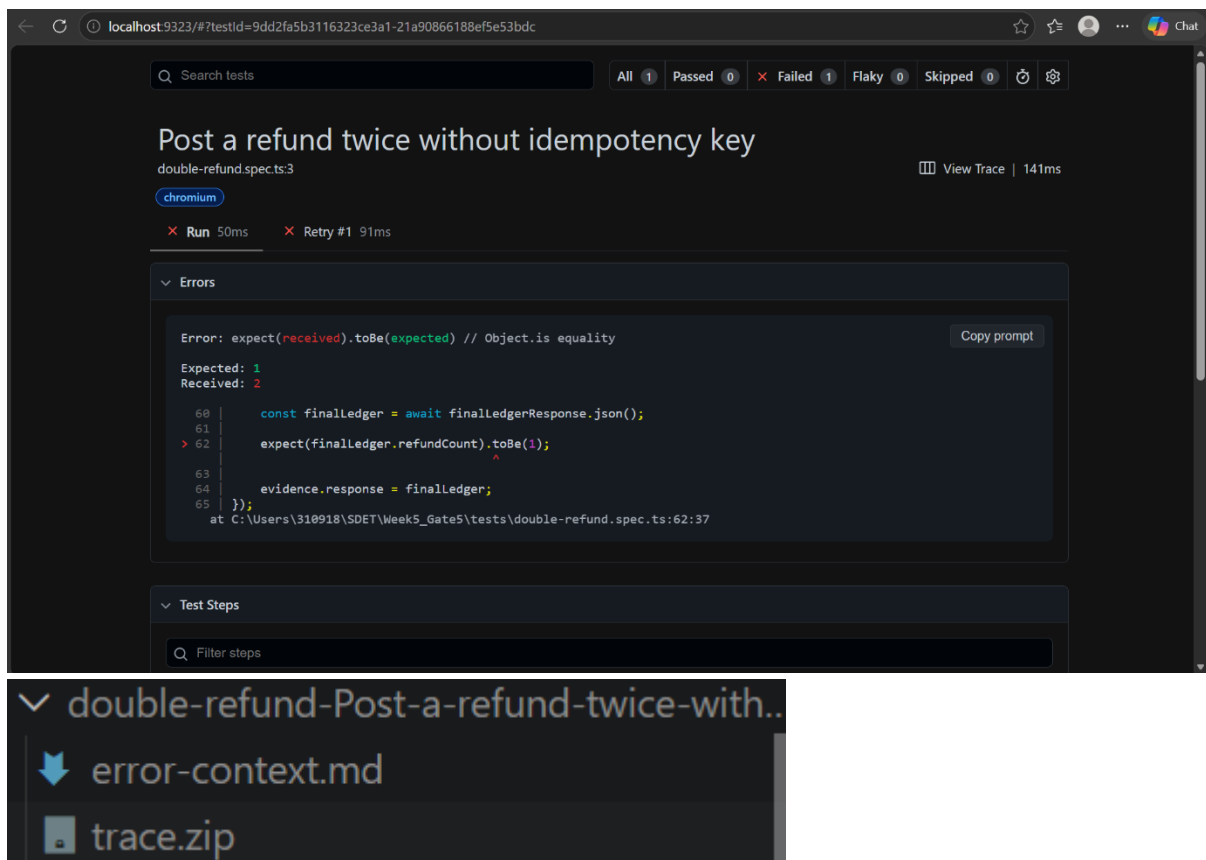
`npx playwright test tests/offline-test-fail.spec.ts`

```
npx playwright test tests/offline-test-fix.spec.ts
```

```
npx playwright show-report
```

3) Double Refund

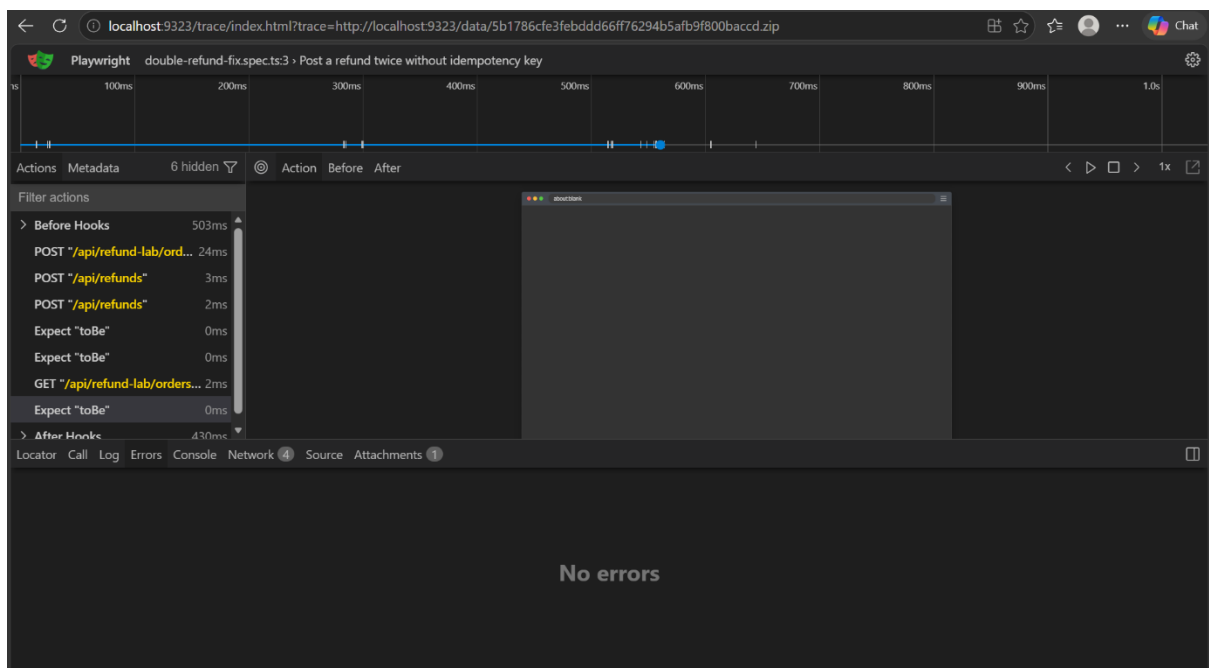
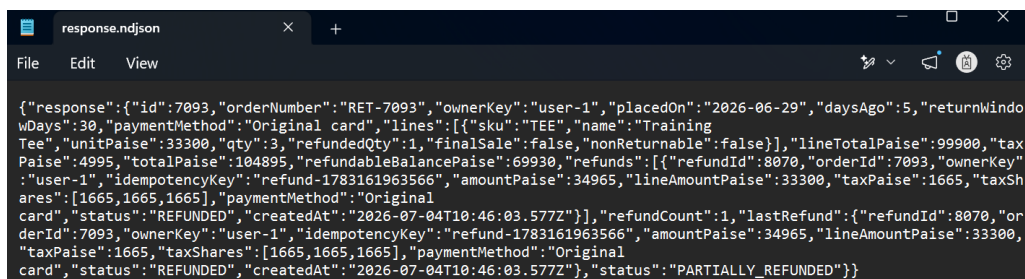
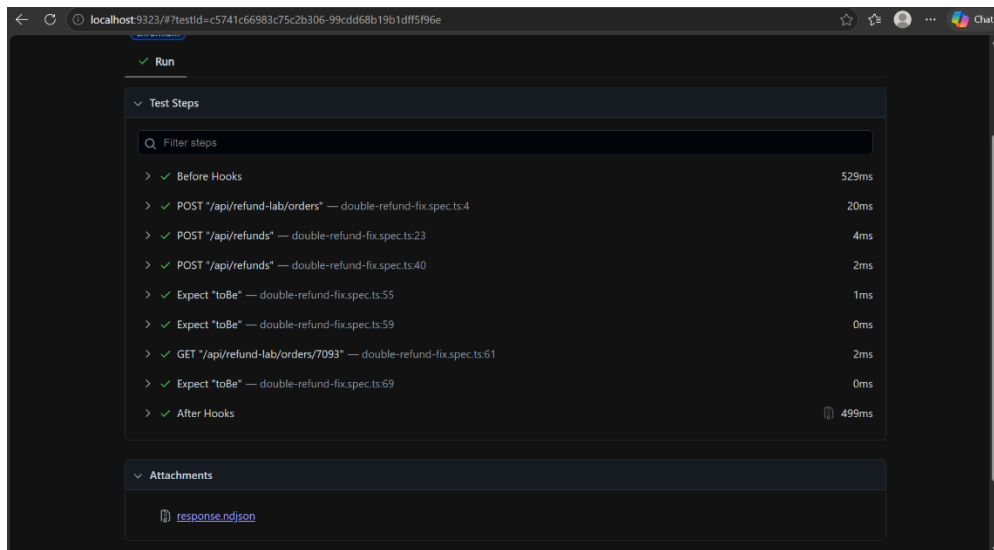
A test file named double-refund.spec.ts is created to post the same refund request twice without idempotency keys to intentionally fail the test. We assert the refund-count to be 1, when in reality it becomes 2 due to the lack of idempotency keys. Failure generates a trace.zip for debugging.



The error is fixed in another test file called double-refund-fix.spec.ts in which we add idempotency keys to the refund request headers to ensure that posting the same refund request twice only results in the creation of a single refund request, ensuring idempotency.

Success generates the refund response body as a json as an artifact attachment in the

report which is downloadable.



Commands:

npx playwright test tests/double-refund.spec.ts

npx playwright test tests/double-refund-fix.spec.ts

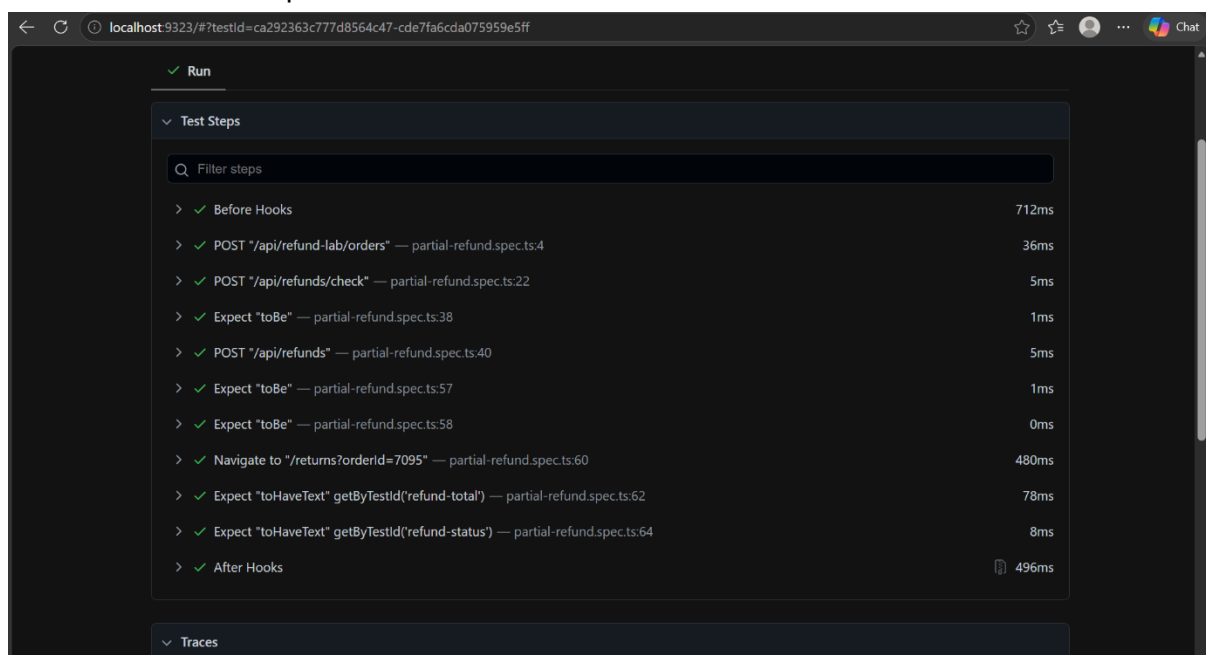
npx playwright show-report

4) Partial Refund

To demonstrate the proper working of partial refunds, a test file called `partial-refund.spec.ts` is created. In the test, an order is seeded in which 3 line items are ordered with a unit price of Rs. 333 and tax of Rs. 49.95. The prices are converted to paise to avoid floating point arithmetic.

Once the order is seeded, refund eligibility is checked to see if we can refund a single item out of the 3 items. After which, we perform the refund and assert the refund amounts to be correct in paise, as well as assert that the refund status is correctly stored as “PARTIALLY_REFUNDED”.

Upon success, a trace file is generated along with the refund response as an artifact attachment in the report.



The screenshot displays the Playwright test runner interface. At the top, the browser address bar shows the URL: `localhost:9323/trace/index.html?trace=http://localhost:9323/data/6c149b7298212aa85965544c186b0411320dc239.zip`. The test title is `partial-refund.spec.ts:3 › Handle a partial refund`. The timeline at the top shows the test duration from 100ms to 1.9s. The left sidebar lists the actions performed during the test:

- Expect "toBe" 1ms
- POST `"/api/refunds"` 4ms
- Expect "toBe" 0ms
- Expect "toBe" 0ms
- Navigate to `"/returns?ord..."` 480ms
- Expect "toHaveText" 78ms
- getByTestId(`refund-total`)
- Expect "toHaveText" 9ms
- getByTestId(`refund-status`)

The main window shows a screenshot of the application under test, which is a "Returns & Refunds" page. The page includes a "Refund Ledger" table and a "Refundable Lines" table. Below the screenshot, the "No errors" message is displayed. At the bottom, the console output shows the JSON response of the test:

```
[{"response":{"refundId":8072,"orderId":7095,"ownerKey":"user-1","idempotencyKey":"a2b86187-3544-469b-8a68-b1c7b85fa7bf","amountPaise":34965,"lineAmountPaise":33300,"taxPaise":1665,"taxShares":[1665,1665,1665],"paymentMethod":"Original card","status":"REFUNDED","createdAt":"2026-07-04T11:06:16.300Z","orderStatus":"PARTIALLY_REFUNDED","refundCount":1,"refundableBalancePaise":69930}}
```

Commands:

`npx playwright test tests/partial-refund.spec.ts`

`npx playwright show-report`